ARBITER Gate 評審可能會問的 Q&A

InnoServe 2026 決賽（11/7）答辯準備 · 依 CLAUDE.md 第 7 節、reports/ 與 AIS3 時期 RESULTS 整理 · 2026-10-28

> **怎麼用這份：**每題先講**綠色的一句話**（約 15 秒），評審追問再講灰線後面的補充。紅字是不要說、或一定要主動說的地方。
> **數字分兩個時期，不能混用：**AIS3 時期是 nemotron-3-ultra-550b 量的；InnoServe（10/28 起）是 arbiter-gpt（gpt-oss 120B）量的。對外說「5 次中 4 次」，不要說「80%」。

最重要的 6 個數字（背起來）

數字	意思	時期 / 模型
9 / 15	repo 模式即時稽核 StakingVault，用 victim_loss 證實的次數	InnoServe / arbiter-gpt
2/0/7 → 2/2/0	真鏈上：原合約受害者拿回 0、攻擊者賺 7 ETH；套 AI 修補後受害者拿回 2、攻擊者 0。連跑 12 次結果完全相同	InnoServe（攻擊是參考攻擊，修補是 arbiter-gpt）
15 / 15	AI 修補後，攻擊全部失效（3 個修補 × 5 個攻擊；但只有 2 種手法）	InnoServe / arbiter-gpt
5 / 5	修補閘門自我測試：5 種真假修補，都被正確接受或擋下，而且是被對的那道檢查擋下	InnoServe（不用模型）
8.7 對 28.5 分鐘	每找到一個真漏洞，分析師要花的時間（ARBITER 對 Bastet）	AIS3 / nemotron
37%	舊判定條件下，68 個被接受的證明裡有 37% 是誤報，所以 7/28 改成「問誰虧了」	AIS3 / nemotron

一、這是什麼、和別人有什麼不同

Q1　用一句話說，你們做了什麼？

智能合約部署前的資安閘門：AI 不准「說」有漏洞，必須「做出」一個攻擊並在 EVM 上執行成功，才擋下部署；擋下時還會附上一個經過驗證的修補。

> 結果分四級：已證實可攻擊（擋部署）/ 靜態工具提示但未證實 / 未發現可證明攻擊（不等於安全）/ 無法判定（稽核未完成）。永遠不說「安全」。

Q2　和 Slither、Mythril 這類靜態分析工具有什麼不同？

靜態工具回報「這裡長得像漏洞」，要人一筆一筆確認；我們回報的每一筆都附一段實際跑成功的攻擊，人不用再驗證真假。

> 代價是找得比較少：EVM 示範不出來的缺陷（例如滑價保護設成 0）我們抓不到。所以它是「補」靜態工具，不是取代。四級判定裡也保留了「靜態工具提示但未證實」這一級給 Slither。

Q3　直接問 ChatGPT「這份合約有沒有漏洞」不就好了？

LLM 的回答是一段文字主張，可能是對的、也可能是編的；我們把「算不算成功」交給 EVM 和系統寫好的條件，模型改不了。

> AIS3 時期量過現成工具 Bastet（56 個 LLM 偵測提示取 OR）：20 份已修好的合約全部被判有漏洞，1,152 個發現沒有一個能實際執行驗證。這就是我們出發的問題。

Q4　這個想法是你們第一個提出的嗎？

不是。讓 AI 寫攻擊來證明漏洞，SmartPoC 等研究已經有類似方向，我們不主張首創。

> 我們的重點在三件事：(1) 判定條件由系統定義，而且花了四個版本修掉 AI 作弊的空間；(2) 把它做成部署前閘門，判定用語不說「安全」；(3) 修補後用**同一個攻擊**反證，加上 5 道檢查。

Q5　使用者實際怎麼用？

打開網頁、上傳一份 .sol，約 2～8 分鐘後拿到結果頁：判定、受害者損失表（A/B/G）、修補 diff、修補後的損失表。

> 另外可以看錄好的結果（不呼叫模型，幾秒）。命令列版 `gate.py` 輸出 JSON 和結束碼（1 = 擋部署、2 = 未完成、0 = 未擋），可以接進 CI。GitHub Action / SARIF 還在開發中。

二、可信度：怎麼知道結果是真的

Q6　AI 寫的攻擊，怎麼確定不是 AI 自己作弊？

AI 只寫攻擊程式，「攻擊成功」的條件是系統另外加進去的，AI 看不到也改不了；而且有 8 支防作弊探針，每次改程式都要全部通過。

> 探針測的是 AI 曾經用過、或可能用的作弊手法：用測試指令憑空生錢、偽造權限、在初始設定裡把自己設成 owner、拿回自己捐的錢當獲利、躲開判定條件等。每支都是正反兩向：真攻擊要接受，假攻擊要拒絕。

Q7　「攻擊成功」的條件到底是什麼？

victim_loss：同一位普通使用者，把「有攻擊」和「沒攻擊」兩種劇本各跑一次；有攻擊時他拿回的錢比較少、而且攻擊者有獲利，才算成立。

> 結果頁上的 A = 沒攻擊時受害者拿回多少、B = 有攻擊時拿回多少、G = 攻擊者獲利。B < A 才代表真的有人受害。這是第 4 版，前三版（測試通過就算、攻擊者有沒有賺、攻擊者比誠實使用者賺得多）都被我們自己找到漏洞而淘汰。

Q8　為什麼要換四個版本？舊的哪裡不好？

因為 AI 一直找得到漏洞鑽：7/28 回頭審查 68 個被接受的證明，37% 是誤報，所以改成問「誰虧了」。

> 例子：一份會發空投的合約，修好的版本攻擊者也「賺錢」；或者基準線是 AI 自己寫的，它就寫一個很弱的基準。換成 victim_loss 之後，AIS3 第二輪誤報從 13 降到 4。這段失敗過程是 AIS3 專題主要的研究內容，我們完整保留紀錄。

Q9　那你們會漏掉漏洞嗎？

會，而且我們刻意在畫面上講清楚：找不到攻擊時只顯示「未發現可證明攻擊（不等於安全）」。

> 漏掉的原因有兩種：模型這次沒寫出成立的攻擊（即時稽核 StakingVault 15 次有 6 次沒證實）；或者那種缺陷本來就很難在單一 EVM 環境示範（例如需要真實市場和交易排序的滑價問題）。

Q10 為什麼不直接說「安全」？使用者不是最想知道這個嗎？

因為我們證明不了「沒有漏洞」，只證明得了「這個攻擊成立」；說「安全」會讓人放心部署一份其實有洞的合約，這比不說更危險。

> 網頁和輸出都做過用語檢查：CLEAR、safe 出現 0 次；「安全」只出現在「不等於安全」這四個字裡。結束碼 0 的說明也寫成「未擋部署（不代表沒有漏洞）」。

Q11 在測試環境成功，到真的鏈上也會成功嗎？

我們有一條「證據階梯」：測試環境 → 匯出成獨立專案別處也能跑 → 本地真鏈上用真的簽章交易重演。demo 會現場展示最後一階。

> 真鏈上沒有測試作弊指令、要真的私鑰、真的 gas。StakingVault 在真鏈上：原合約 2/0/7（受害者 2 ETH 被拿走），套 AI 修補後 2/2/0，數字和測試環境完全一致，連跑 12 次交易 hash 都一樣。

一定要主動說：真鏈上的攻擊是 7/28 為了校正判定寫的已知參考攻擊（DRAIN），不是模型寫的；修補才是 AI 產生的。

Q12 為什麼真鏈上不用 AI 自己寫的攻擊？

因為 AI 這次寫的攻擊比較弱，只重入一次，要連打 8 輪才傷得到別人；我們的自動上鏈工具一次只重播一輪，所以上鏈後不成立，我們就照實換成已知攻擊來示範修補的效果。

> 這反而說明為什麼要測 4 種環境（預設、受害者餘額 1000 倍、一週後、8 輪攻擊）：AI 的攻擊在前 3 種環境都沒讓受害者受損（2/2/1），只有 8 輪時是 2/0/7。只測一種環境就會誤判它「沒有危害」。

三、數字與成績

Q13 準確率多少？

分兩個時期講。AIS3 時期 40 份配對樣本：F1 0.5455，輸給調校過的 Bastet（0.6909）；我們的強項是特異度 0.80，以及每個真漏洞只要分析師 8.7 分鐘（Bastet 28.5 分鐘）。

> InnoServe 換成 arbiter-gpt 後，還沒有重跑整份評測集，只在示範合約上量可靠度：StakingVault 即時稽核 15 次證實 9 次。這兩組數字不能混在一起比。

F1 輸給 Bastet 要主動講，不要等評審發現。

Q14 F1 輸人家，那你們的價值在哪？

F1 把「說中」和「證明」算成一樣；我們每一個陽性都附可執行的攻擊，分析師不用再驗證，所以每個真漏洞的人力成本是對方的三分之一不到。

> 出廠設定的 Bastet 特異度是 0（修好的合約也全部判有漏洞），F1 0.667 剛好等於「永遠回答有漏洞」的分數。調校後它贏我們，這點我們承認，也寫在報告裡。

Q15 樣本才 40 份，數字有統計意義嗎？

沒有到顯著，我們也照這樣寫：AIS3 第二輪所有差異的信賴區間都包含 0，依事先登記的標準判定為「不算贏」。

> 同設定跑 3 次，MCC 在 0.12～0.30 之間跳動，所以只引用範圍。實驗是事先登記的（PREREGISTRATION），也找人從反方審查，12 項指控 11 項成立，全部修正並記錄。

Q16 評測集是怎麼來的？會不會自己出題自己考？

每一對樣本都要先通過機器入場檢查：參考攻擊在有漏洞版成功、在修好版失敗、兩版都能編譯，全由 forge 驗證，沒有模型參與。

> 主要評測集是自己寫的 20 種漏洞 × 2 = 40 份，每對只差 1～4 行。原本拿到的 14 個樣本多半是無法編譯的片段，沒辦法拿來跑攻擊。自己寫的風險確實存在，所以 F1 我們不拿來宣稱勝出。

Q17 現場即時稽核只有 9 / 15，不會太不穩嗎？

會不穩，所以 demo 主線用錄好的結果和真鏈重播，這兩段每次結果都一樣；即時稽核是加分項，失敗也照實顯示「未發現可證明攻擊」。

> 重點是失敗的方向：沒證實時它不會誤報「有漏洞」，也不會說「安全」。9 / 15 是 arbiter-gpt 在 repo 模式累計的數字（10/28），11/3 會再做一次 10 次的正式可靠度測試。

四、修補

Q18 AI 寫的修補，會不會把合約改壞？

修補要依序通過 5 道檢查才接受：原攻擊還能重現 → 最多改 12 行且不准用投機手法 → 介面不變 → 同一個攻擊在 4 種環境都失效 → 正常使用者的錢不能變少。

> 5 道檢查都是 forge 實際執行，不靠模型判斷。自我測試用 5 種修補：2 種真修補接受；「一律 revert」被第 2 道擋（正常人也領不到錢）；「刪掉函式」被第 3 道擋；「只准外部帳戶呼叫」被修改檢查擋（只是把這個攻擊者擋在門外）。

Q19 通過修補閘門，就代表合約安全了嗎？

不代表。畫面上寫的是「修補已通過驗證（針對此攻擊）」：這個攻擊失效、正常用法沒壞、介面沒變，其他漏洞不保證。

> 我們另外做了交叉驗證：3 個修補 × 5 個攻擊，15 次全部失效，其中 DRAIN 是產生修補時模型沒看過的攻擊（3 / 3 失效）。但只有 2 種手法、都是 unstake() 重入。修補是建議，要由開發者審閱後套用並重新稽核。

不要說「AI 修好了合約」。

Q20 為什麼限制最多改 12 行？

修補越大越難審閱，也越容易偷偷改掉別的行為；漏洞通常只差幾行（我們的評測集每對只差 1～4 行）。

> 修改檢查還禁止：檢查呼叫者身分、寫死地址、selfdestruct、inline assembly、無條件 revert。這些都是「讓這個攻擊失敗、但沒有真的修好」的常見做法。

五、AIS3 與 InnoServe：哪裡是新的

Q21 這是 AIS3 的作品，InnoServe 有什麼明顯改善？

AIS3 做出的是研究用的稽核核心；InnoServe 把它延伸成部署前閘門：四級判定、受害者損失表、AI 修補加 5 道驗證、網頁、命令列閘門、真鏈 demo，全部是新寫的。

> AIS3 來源已在系統概述文件揭露。判定核心（`agent.py`、`tools.py`）和 InnoServe 起點相比零差異，新功能都放在新檔案（`gate.py`、`patch.py`、`verdict.py`、`web.py`），可以用 git 直接驗證。

Q22 為什麼核心不改？不是可以改得更好嗎？

核心的判定規則是 AIS3 時期花了四個版本、配上 8 支探針才穩定下來的；在不改它的前提下加功能，才能說新的結果還是同一套可信的判定。

> 我們確實找到一個核心的舊缺陷想留到決賽後再修，見 Q27。

六、實用性與落地

Q23 一次要多久？多少錢？

即時稽核加修補約 2～8 分鐘（實測 125～465 秒）；找到攻擊時多半 2～3 分鐘，沒找到時比較久。模型跑在自己的 DGX Spark 上，不需要付 API 費用。

> 一次約 4～7 個模型請求。錄好的結果重新驗證只要 1～2 秒、0 個請求；真鏈重播約 13 秒。

Q24 用哪個模型？為什麼不用 GPT 或 Claude？

gpt-oss 120B，跑在我們自己的 DGX Spark 上：非陸資模型（符合競賽規則）、不花錢、合約程式碼不會送出去、決賽可以離線展示。

> 架構不綁特定模型：模型只負責「提出」攻擊和修補，對錯由系統判定，換更強的模型通常只會提高找到的比例，不會改變判定的可信度。Claude API 評估過當備案，但要另寫轉接程式，也要先測資安類請求會不會被拒答。

Q25 只能測單一檔案嗎？真實專案很多檔案怎麼辦？

核心支援 repo 模式：在原專案的編譯環境裡稽核，import 和相依套件都能解析，模型也能讀其他檔案。網頁上傳目前限沒有 import 的單一檔案。

> AIS3 時期量過：真實稽核 repo 在單檔模式下 24 份能編譯 0 份，所以才做了 repo 模式。修補閘門目前也限沒有 import 的檔案。

Q26 怎麼接進開發流程？

命令列 `gate.py` 已經輸出標準的 JSON 和結束碼，CI 可以直接用結束碼決定要不要擋合併；SARIF 輸出和 GitHub Action 在開發中。

> 另一個規劃中的功能是「回歸測試下載」：把證實的攻擊匯出成 Foundry 測試檔，放進開發者自己的專案，以後有人改壞會自動失敗。核心的匯出功能（`dump.py`）AIS3 時期已經有了。

七、限制、缺陷與倫理

Q27 系統有沒有你們知道、但還沒修的問題？

有，我們主動說：核心工具說明裡還留著已經退休的舊判定條件，單檔模式下模型會選舊條件（5 次都是），所以我們把網頁上傳改成 repo 模式，並在畫面上照實標示「寬鬆謂詞」。

> 改成 repo 模式前，事先寫好切換條件（5 次都選 victim_loss 才切換），結果 5 / 5 符合才切換。核心決賽前不改，留到決賽後修。其他還沒做的：沙箱隔離、即時進度、網頁內的鏈上重播頁、SARIF，畫面上都標「開發中」。

Q28 上傳的合約直接在你們機器上編譯執行，不危險嗎？

目前沒有沙箱，所以網頁只綁 127.0.0.1，不對外開放；沙箱隔離是對外服務前一定要做的事，畫面上也標了「開發中」。

Q29 一個會自動寫攻擊程式的工具，不會被壞人拿去用嗎？

它是部署前給開發者用的：ARBITER Gate 只在本地測試環境和本地鏈上執行，不對真實主網送交易；而且修補和攻擊一起產生，價值在「部署前修掉」。

> 資料方面，評測集裡尚未完成廠商揭露的第三方漏洞程式碼已經去識別化、不公開（9/26）。demo 用的 StakingVault 是我們自己寫的範例合約，沒有部署在任何鏈上。

Q30 只支援以太坊 / Solidity 嗎？

目前是 Solidity 和 EVM，所以 EVM 相容鏈（以太坊、L2、BSC 等）都適用；其他虛擬機需要換掉執行層，但「系統定義成功條件、AI 只負責提出」的架構可以沿用。

八、demo 現場可能的追問

Q31 剛剛即時上傳那筆沒有證實，是系統壞了嗎？

沒有壞，是模型這次沒寫出成立的攻擊。arbiter-gpt 在 repo 模式累計 15 次證實 9 次，大約三次有一次不成功，所以主線用錄好的結果；它沒證實時也不會說「安全」。

Q32 錄好的結果，怎麼證明不是事後改過的？

錄好的是 AI 寫的攻擊程式和修補；按下「看結果」時，系統會用 forge 當場重新執行攻擊、重新量 A/B/G、重新跑修補的 5 道檢查，不是播放截圖。

> 這一步 0 個模型請求，約 1～2 秒，結果頁上會顯示「本次 LLM 請求數 0」。

Q33 表格前三個環境攻擊者賺 1 ETH，受害者卻沒損失，那錢從哪來？

來自合約裡其他人的資金：部署時存入的 5 ether。受害者這位使用者沒受損，所以前三個環境不算成立；第 4 個環境（8 輪攻擊）受害者拿回 0，才是 2/0/7。

> 我們照實顯示，不為了好看只秀成立的那一格。

Q34 修補版的攻擊交易為什麼沒有交易 hash？

因為修補後攻擊一定會 revert，節點在估算 gas 時就拒絕了，交易沒有送出上鏈；受害者那筆正常提款有 hash，拿回 2 ETH。

不要說、不要做：

- 不要說「安全」「CLEAR」「通過 = 沒漏洞」「80%」。
- 不要把真鏈上的攻擊說成模型寫的；不要說修補「修好了合約」。
- 不要把 AIS3（nemotron）和 InnoServe（arbiter-gpt）的數字放在同一句比較。
- 不要說「第一個」「首創」。
- 「數十億美元損失」這類市場數字，**要附出處才講**（例如 Chainalysis 年度報告，引用前先查證年份與數字）；也要注意很多損失來自私鑰外洩，不是合約漏洞，不要全算到合約頭上。